Program Studi Teknik Informatika
Institut Teknologi Sumatera

MODUL PRAKTIKUM

Sistem Operasi

IF25-12007 | Genap 2025/2026

Mata Kuliah	Sistem Operasi (IF25-12007)
Semester	Genap 2025/2026
Institusi	Institut Teknologi Sumatera

© 2026 — Institut Teknologi Sumatera

TIM PENGAJAR & ASISTEN

Dosen Pengampu

Martin Clinton Tosima Manullang, S.T., M.T., Ph.D.

Ilham Firman Ashari, S.Kom., M.T

I Wayan Wiprayoga Wisesa, S.Kom., M.Kom.

Eko Dwi Nugroho, S.Kom., M.Cs.

Asisten Praktikum

Ahmat Prayoga Sembiring	123140053
Andryano Shevchenko Limbong	123140205
Aziz Kurniawan	122140097
Dito Rifki Irawan	122140153
Ebentua Philippus Limbong	123140086
Elsa Elisa Yohana Sianturi	122140135
Fathan Andi Kartagama	122140055
Miftahul Khoiriyah	123140064
Muhammad Bintang Al-Fasya	123140098
Muhammad Fadhil Zurani	122140146
Reza Chairul Manam	120140086
Stevanus Cahya Anggara	123140038
Varasina Farmadani	123140107
Zakhi Algifari	122140198

Contents

1 Instalasi Ubuntu pada VirtualBox **5**

1.1 Tugas 5

2 Automasi dengan Bash Scripting **7**

2.1 Tugas 7

3 Otomatisasi Proses Build dengan Makefile **9**

3.1 Tugas 9

4 Implementasi Multi-threading **11**

4.1 Tujuan 11

4.2 Sesi 1 — Persiapan Lingkungan Kerja 11

4.2.1 Masuk ke Container Docker 11

4.3 Sesi 2 — Implementasi `thread_create()` 13

4.3.1 Thread Control Block dan Layout Memori 13

4.3.2 Implementasi Fungsi `thread_create()` 14

4.4 Sesi 3 — Implementasi *Cooperative Round-Robin* 15

4.4.1 Konsep Status Thread 15

4.4.2 Implementasi Fungsi *Scheduling* 16

4.5 Sesi 4 — Kompilasi dan Pengujian QEMU 17

4.6 Eksplorasi Mandiri 18

5 Implementasi Mutex dan Locks **21**

5.1 Tugas 21

6 Penjadwalan Proses Berbasis Prioritas 23

6.1 Tugas 23

CHAPTER 1

Instalasi Ubuntu pada VirtualBox

1.1 Tugas

Jawablah pertanyaan-pertanyaan berikut berdasarkan hasil praktikum Anda. Sertakan bukti berupa *screenshot* atau penggalan keluaran terminal yang relevan pada setiap jawaban.

1. **[Topik Pertanyaan 1]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   Jawaban:
   [Isi jawaban di sini]

2. **[Topik Pertanyaan 2]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   (a) Sub-pertanyaan a.
   Jawaban: *[Isi jawaban di sini]*

   (b) Sub-pertanyaan b.
   Jawaban: *[Isi jawaban di sini]*

3. **[Topik Pertanyaan 3]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   Jawaban:
   [Isi jawaban di sini]

4. **[Eksplorasi Mandiri]**

TODO: ganti dengan pertanyaan eksplorasi sesungguhnya.

Jawaban:

[Isi jawaban di sini]

CHAPTER 2

Automasi dengan Bash Scripting

2.1 Tugas

Jawablah pertanyaan-pertanyaan berikut berdasarkan hasil praktikum Anda. Sertakan bukti berupa *screenshot* atau penggalan keluaran terminal yang relevan pada setiap jawaban.

1. **[Topik Pertanyaan 1]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   Jawaban:
   [Isi jawaban di sini]

2. **[Topik Pertanyaan 2]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   (a) Sub-pertanyaan a.
   Jawaban: *[Isi jawaban di sini]*

   (b) Sub-pertanyaan b.
   Jawaban: *[Isi jawaban di sini]*

3. **[Topik Pertanyaan 3]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   Jawaban:
   [Isi jawaban di sini]

4. **[Eksplorasi Mandiri]**

TODO: ganti dengan pertanyaan eksplorasi sesungguhnya.

Jawaban:

[Isi jawaban di sini]

CHAPTER 3

Otomatisasi Proses Build dengan Makefile

3.1 Tugas

Jawablah pertanyaan-pertanyaan berikut berdasarkan hasil praktikum Anda. Sertakan bukti berupa *screenshot* atau penggalan keluaran terminal yang relevan pada setiap jawaban.

1. **[Topik Pertanyaan 1]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   Jawaban:
   [Isi jawaban di sini]

2. **[Topik Pertanyaan 2]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   (a) Sub-pertanyaan a.
   Jawaban: *[Isi jawaban di sini]*

   (b) Sub-pertanyaan b.
   Jawaban: *[Isi jawaban di sini]*

3. **[Topik Pertanyaan 3]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   Jawaban:

[Isi jawaban di sini]

4. **[Eksplorasi Mandiri]**
   TODO: ganti dengan pertanyaan eksplorasi sesungguhnya.

   Jawaban:
   [Isi jawaban di sini]

CHAPTER 4

Implementasi Multi-threading

4.1 Tujuan

Jawablah pertanyaan-pertanyaan berikut berdasarkan hasil praktikum Anda sesuai urutan sesi pada Modul 8. Sertakan bukti berupa *screenshot* atau penggalan keluaran terminal yang relevan pada setiap jawaban.

4.2 Sesi 1 — Persiapan Lingkungan Kerja

4.2.1 Masuk ke Container Docker

Jalankan perintah berikut di terminal, kemudian masuk ke direktori *workspace*:

```
docker run --rm -it \
    -v "C:\path\ke\starter-code:/workspace" \
    sinanonym/ifos-env
cd /workspace
```

Kode 4.1: Menjalankan container dan masuk workspace

1. **Compile starter code sebelum implementasi.**
   Jalankan perintah berikut di dalam container:

```
make clean && make

```

Kode 4.2: Perintah kompilasi awal

Tuliskan seluruh *warning* yang muncul pada saat kompilasi dan jelaskan mengapa *warning* tersebut muncul pada kode *stub*.

Jawaban:
[Isi jawaban di sini]

2. **Jalankan QEMU sebelum implementasi.**
Jalankan perintah berikut lalu sertakan *screenshot* layar QEMU. Jelaskan mengapa sistem berhenti di sana dan belum menampilkan output Thread A/B/C.

```
make run

```

Kode 4.3: Menjalankan emulator QEMU

Screenshot:

[Tempel screenshot layar QEMU di sini]

Gambar 4.1: Tampilan QEMU sebelum implementasi

Jawaban:
[Isi jawaban di sini]

4.3 Sesi 2 — Implementasi `thread_create()`

4.3.1 Thread Control Block dan Layout Memori

1. **Field-field `struct thread`.**
   Perhatikan definisi `struct thread` di `src/thread.h`. Isilah tabel berikut berdasarkan kode sumber.

Tabel 4.1: Field-field pada `struct thread`

Field	Tipe Data	Fungsi
`tid`	`tid_t`	*[Isi]*
`status`	`enum thread_status`	*[Isi]*
`name`	`char[16]`	*[Isi]*
`stack`	`uint8_t *`	*[Isi]*
`priority`	`int`	*[Isi]*
`magic`	`unsigned`	*[Isi]*

2. **Layout halaman 4 KB setiap thread.**
   Setiap thread menempati satu halaman (*page*) memori berukuran 4 KB. Isilah kolom *Isi* pada tabel layout di bawah ini untuk menggambarkan posisi `struct thread`, *stack*, dan ketiga frame yang disiapkan oleh `thread_create()`.

Tabel 4.2: Layout halaman 4 KB milik sebuah thread

Offset (relatif)	Isi
`+4096`	*[Isi: puncak halaman / ESP awal]*
`...`	`switch_threads_frame`
`...`	`switch_entry_frame`
`...`	`kernel_thread_frame`
`...`	*[Isi: area stack kosong, tumbuh ke bawah (↓)]*
`+0`	`struct thread` (TCB)

3. **Analisis fungsi `running_thread()`.**
   Perhatikan potongan kode berikut:

```
struct thread *running_thread(void) {
    uint32_t esp;
    __asm__ volatile ("mov %%esp, %0" : "=g" (esp));
    return (struct thread *)(void *)(esp & ~(uint32_t)(PGSIZE
   - 1));
}
```

```
6
```

Kode 4.4: Implementasi `running_thread()` di `src/thread.c`

(a) Jelaskan apa yang dilakukan operasi bitwise `esp & ~(PGSIZE - 1)` dan mengapa hasilnya selalu merupakan alamat awal *page*.

Jawaban:
[Isi jawaban di sini]

(b) Mengapa teknik ini hanya benar jika `struct thread` selalu berada di offset 0 dari halamannya? Apa dampaknya jika `initial_thread_page` di `boot.asm` tidak di-*align* ke 4096 byte?

Jawaban:
[Isi jawaban di sini]

(c) Apa fungsi `THREAD_MAGIC` dan bagaimana `thread_current()` menggunakannya untuk mendeteksi *stack overflow*?

Jawaban:
[Isi jawaban di sini]

4.3.2 Implementasi Fungsi `thread_create()`

1. **Tiga stack frame pada `thread_create()`.**
   Isilah tabel berikut berdasarkan komentar petunjuk di `src/thread.c`.

Tabel 4.3: Tiga stack frame yang disiapkan `thread_create()`

Urutan push	Nama frame	Field penting	Nilai yang diisi
1 — paling atas	`kernel_thread_frame`	`eip`, `function`, `aux`	*[Isi]*
2 — tengah	`switch_entry_frame`	`eip`	*[Isi]*
3 — paling bawah	`switch_threads_frame`	`eip`, register *callee-saved*	*[Isi]*

2. **Alur eksekusi pertama thread baru.**
   Saat thread baru **pertama kali** dijadwalkan melalui `switch_threads()`, jelaskan secara berurutan apa yang terjadi mulai dari `switch_entry` hingga fungsi thread (`func_a`, `func_b`, atau `func_c`) benar-benar mulai berjalan.

Jawaban:

[Isi jawaban di sini]

3. **Tulis implementasi `thread_create()`.**
   Salin kode yang telah Anda tulis di `src/thread.c`:

```
tid_t thread_create(const char *name, int priority,
                    thread_func *function, void *aux) {

    /* Tulis implementasi Anda di sini */

}

```

Kode 4.5: Implementasi `thread_create()` oleh mahasiswa

4. **Penanganan kegagalan alokasi.**
   Kapan `allocate_thread_page()` dapat mengembalikan `NULL`, dan bagaimana `thread_create()` harus menanganinya?

   Jawaban:

   [Isi jawaban di sini]

4.4 Sesi 3 — Implementasi *Cooperative Round-Robin*

4.4.1 Konsep Status Thread

1. **Perbedaan `THREAD_READY` dan `THREAD_BLOCKED`.**
   Lengkapi tabel perbandingan berikut.

Tabel 4.4: Perbandingan status `THREAD_READY` dan `THREAD_BLOCKED`

Aspek	THREAD_READY	THREAD_BLOCKED
Siapa yang menetapkan?	*[Isi]*	*[Isi]*
Masuk ke *ready queue*?	*[Isi]*	*[Isi]*
Cara kembali ke `RUNNING`	*[Isi]*	*[Isi]*
Contoh penggunaan	*[Isi]*	*[Isi]*

4.4.2 Implementasi Fungsi *Scheduling*

1. **Tulis implementasi `thread_yield()`.**
   Salin kode yang telah Anda tulis di `src/thread.c`:

```c
void thread_yield(void) {

    /* Tulis implementasi Anda di sini */

}

```

Kode 4.6: Implementasi `thread_yield()` oleh mahasiswa

2. **Tulis implementasi `next_thread_to_run()` (Round-Robin FIFO).**
   Salin kode yang telah Anda tulis di `src/thread.c`:

```c
struct thread *next_thread_to_run(void) {

    /* Tulis implementasi Anda di sini */

}

```

Kode 4.7: Implementasi `next_thread_to_run()` oleh mahasiswa

3. **Mengapa mengembalikan `initial_thread`, bukan `NULL`?**
   Jelaskan apa yang akan terjadi pada fungsi `schedule()` jika `next_thread_to_run()` mengembalikan `NULL` saat antrian kosong.

   Jawaban:
   [Isi jawaban di sini]

4. **`thread_block()` vs. `thread_yield()` di akhir `kernel_main`.**
   Jelaskan perbedaan perilaku kedua fungsi tersebut setelah Thread A, B, dan C berhasil dibuat, dan mengapa `thread_block()` adalah pilihan yang benar.

   Jawaban:
   [Isi jawaban di sini]

4.5 Sesi 4 — Kompilasi dan Pengujian QEMU

1. **Compile ulang setelah implementasi selesai.**

```
make clean && make && make run

```

Kode 4.8: Perintah build dan run setelah implementasi

Sertakan *screenshot* layar QEMU yang menampilkan ketiga thread berjalan bergantian.

[Tempel screenshot layar QEMU setelah implementasi di sini]

Gambar 4.2: Tampilan QEMU setelah implementasi berhasil

2. **Verifikasi urutan penjadwalan.**
   Ekspektasi keluaran sistem yang benar adalah sebagai berikut:

```
===================================
  IF-OS  |  Praktikum Sistem Operasi
===================================

Inisialisasi sistem thread...
Membuat thread A, B, C...
Memulai scheduling...

[Thread A] tick 0
[Thread B] tick 0
```

```
[Thread C] tick 0
[Thread A] tick 1
[Thread B] tick 1
[Thread C] tick 1
...

```

Kode 4.9: Ekspektasi output Round-Robin di layar QEMU

Apakah urutan Thread A → Thread B → Thread C → Thread A → ... terjaga secara konsisten pada hasil Anda? Jelaskan mengapa urutan ini dapat dijamin oleh implementasi FIFO pada *ready queue*.

Jawaban:
[Isi jawaban di sini]

3. **Analisis *warning* compiler sebelum dan setelah implementasi.**
   Apakah *warning* `unused-function` dan `unused-variable` yang muncul sebelum implementasi masih ada setelah Anda mengimplementasikan ketiga fungsi? Jelaskan mengapa *warning* tersebut hilang.

   Jawaban:
   [Isi jawaban di sini]

4.6 Eksplorasi Mandiri

1. **Menambah Thread keempat.**
   Tambahkan Thread D dengan fungsi dan warna VGA berbeda di `src/kernel.c`, kemudian compile ulang dan jalankan.

   (a) Sertakan *screenshot* layar QEMU yang menampilkan empat thread berjalan bergantian.

[Tempel screenshot empat thread di sini]

Gambar 4.3: Tampilan QEMU dengan empat thread

(b) Berapa nilai `MAX_THREADS` saat ini? Apa yang terjadi jika Anda membuat thread melebihi batas tersebut? Tunjukkan dengan percobaan dan sertakan bukti.

Jawaban:
[Isi jawaban di sini]

2. **Simulasi *finite thread*.**
Ubah `func_a` di `src/kernel.c` agar hanya mencetak **5 kali** lalu berhenti (ganti `while(1)` dengan loop terbatas). Amati apakah Thread B dan Thread C tetap berjalan setelah Thread A selesai, lalu jelaskan peran `thread_exit()` dalam skenario ini.

Jawaban:
[Isi jawaban di sini]

3. **Refleksi: *cooperative* vs. *preemptive scheduling*.**
Sistem IF-OS saat ini menggunakan *cooperative scheduling* di mana thread hanya berganti saat secara sadar memanggil `thread_yield()`.

(a) Apa kelemahan utama *cooperative scheduling* dibandingkan *preemptive scheduling*? Berikan contoh skenario nyata di mana kelemahan ini berbahaya.

Jawaban:
[Isi jawaban di sini]

(b) Komponen perangkat keras apa yang perlu ditambahkan agar kernel dapat memaksa pergantian thread secara otomatis tanpa bergantung

pada kepatuhan thread itu sendiri?

Jawaban:

[Isi jawaban di sini]

CHAPTER 5

Implementasi Mutex dan Locks

5.1 Tugas

Jawablah pertanyaan-pertanyaan berikut berdasarkan hasil praktikum Anda. Sertakan bukti berupa *screenshot* atau penggalan keluaran terminal yang relevan pada setiap jawaban.

1. **[Topik Pertanyaan 1]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   Jawaban:
   [Isi jawaban di sini]

2. **[Topik Pertanyaan 2]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   (a) Sub-pertanyaan a.
   Jawaban: *[Isi jawaban di sini]*

   (b) Sub-pertanyaan b.
   Jawaban: *[Isi jawaban di sini]*

3. **[Topik Pertanyaan 3]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   Jawaban:
   [Isi jawaban di sini]

4. **[Eksplorasi Mandiri]**

TODO: ganti dengan pertanyaan eksplorasi sesungguhnya.

Jawaban:

[Isi jawaban di sini]

CHAPTER 6

Penjadwalan Proses Berbasis Prioritas

6.1 Tugas

Jawablah pertanyaan-pertanyaan berikut berdasarkan hasil praktikum Anda. Sertakan bukti berupa *screenshot* atau penggalan keluaran terminal yang relevan pada setiap jawaban.

1. **[Topik Pertanyaan 1]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   Jawaban:
   [Isi jawaban di sini]

2. **[Topik Pertanyaan 2]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   (a) Sub-pertanyaan a.
   Jawaban: *[Isi jawaban di sini]*

   (b) Sub-pertanyaan b.
   Jawaban: *[Isi jawaban di sini]*

3. **[Topik Pertanyaan 3]**
   TODO: ganti dengan pertanyaan sesungguhnya.

   Jawaban:

[Isi jawaban di sini]

4. **[Eksplorasi Mandiri]**
   TODO: ganti dengan pertanyaan eksplorasi sesungguhnya.

   Jawaban:
   [Isi jawaban di sini]